

Semana 5
ISIS-1611: Inteligencia Artificial

Carla González

3 de julio de 2026

1. Repaso General Examen 2 Inteligencia Artificial

Las decisiones optimas en juegos se componen de:

1. Búsqueda Minimax: arbol de espacio de estados, Jugador alterna turnos y tiene mejor utilidad alcanzable contra un jugador racional (óptimo)

```

1 def max-value(estado):
2     inicializar = -infinito
3     para cada sucesor del estado:
4         v = max(v, min-value(sucesor))
5     return v

```

```

1 def min-value(estado):
2     inicializar = +infinito
3     para cada sucesor del estado:
4         v = min(v, max-value(sucesor))
5     return v

```

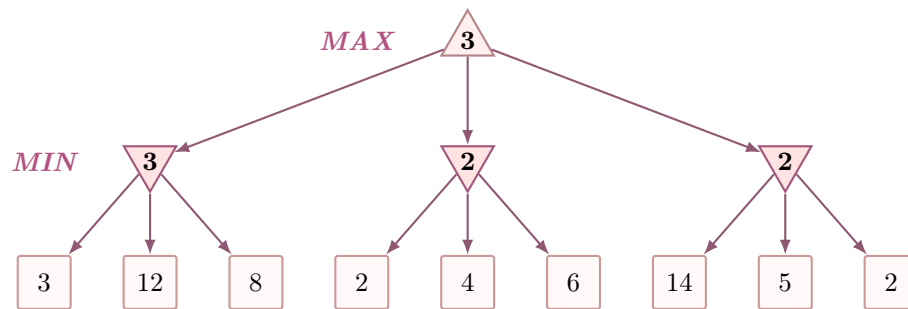


Figura 1: Árbol de búsqueda Minimax: el nodo MAX (raíz) elige el sucesor con mayor valor entre los mínimos que garantizan los nodos MIN; los valores se propagan de las hojas hacia arriba.

1.1. Probabilidad marginal, conjunta y condicional

Definición 1.1. Probabilidad conjunta: $P(A,B)$ es la probabilidad de que ocurran simultáneamente los eventos A y B. Se representa como una tabla donde cada celda es $P = (A = a, B = b)$ y la suma de todas las celdas es 1.

Definición 1.2. Probabilidad marginal: Se obtiene sumando sobre la variable que se quiere eliminar:

$$P(A = a) = \sum_b P(A = a, B = b)$$

En una tabla, corresponde a sumar la fila o columna correspondiente.

Definición 1.3. Probabilidad condicional: La fracción de los casos donde ocurre B es los que también ocurre A.

$$P(A|B) = \frac{P(A, B)}{P(B)}, P(B) > 0$$

Se presente una asimetría importante donde $P(A|B) \neq P(B|A)$ en general.

1.2. Regla del producto y regla de la cadena

Teorema 1.1. Regla del producto:

$$P(A, B) = P(A|B)P(B) = P(B|A)P(A)$$

Teorema 1.2. Regla de la cadena:

$$P(x_1, \dots, x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)\dots P(x_n|x_1, \dots, x_{n-1})$$

Sin ningun supuesto de independencia, el numero de parametros libres crece exponencialmente con n . Cada supuesto de independencia condicional que se agrega elimina variables de la lista de condicionales y reduce ese numero.

1.3. Ejercicio ScoreTrack

ScoreTrack es una plataforma de analisis de rendimiento deportivo. Registra si un partido terminó en victoria (R=victoria) o derrota (R=derrota) segun si el equipo jugó de local (C=local) o visitante(C=visitante). La tabla conjunta $P(R, C)$ reúne datos de 200 partidos de la ultima temporada.

$P(R, C)$	$C = \text{local}$	$C = \text{visitante}$	$P(R)$
$R = \text{victoria}$	0.42	0.18	?
$R = \text{derrota}$	0.28	0.12	?
$P(C)$	?	?	1.00

Cuadro 1: Distribución conjunta $P(R, C)$ con marginales

1. Complete las marginales $P(R)$ y $P(C)$ de la tabla.
2. Calcule $P(C = \text{local} \mid R = \text{victoria})$ usando la definición de probabilidad condicional. Muestre numerador y denominador.
3. Calcule $P(R = \text{victoria} \mid C = \text{local})$. Compare este resultado con el del punto 2. ¿Son iguales? ¿Por qué no?
4. Verifique su respuesta del punto 2 usando la Regla de Bayes: $P(C=\text{local}|R=\text{victoria}) = P(R=\text{victoria}|C=\text{local}) \cdot P(C=\text{local}) / P(R=\text{victoria})$.
5. Calcule $P(R = \text{derrota} \mid C = \text{visitante})$.

2. Procesos de Decisión de Markov (MDPs)

2.1. Definición y propiedad de Markov

Definición 2.1 (MDP). Un MDP se define como la tupla (S, A, P, R, γ) :

- S : conjunto de estados.
- A : conjunto de acciones disponibles.
- $P(s'|s, a)$: modelo de transición, probabilidad de llegar a s' al ejecutar a en s .
- $R(s, a, s')$: función de recompensa asociada a esa transición.
- $\gamma \in [0, 1]$: factor de descuento, penaliza recompensas lejanas en el tiempo.

Definición 2.2 (Propiedad de Markov).

$$P(s_{t+1} \mid s_t, a_t) = P(s_{t+1} \mid s_0, a_0, \dots, s_t, a_t)$$

El estado siguiente depende únicamente del estado y la acción actuales, no de todo el historial que llevó hasta ahí.

2.2. Política, utilidad y ecuación de Bellman

Definición 2.3 (Política y utilidad). Una **política** $\pi(s)$ asigna una acción a cada estado. La **utilidad** de una secuencia de estados es el retorno descontado $U = \sum_{t=0}^{\infty} \gamma^t R_t$. La **política óptima** π^* es la que maximiza la utilidad esperada desde cualquier estado inicial.

Teorema 2.1 (Ecuación de Bellman).

$$U^*(s) = \max_a \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma U^*(s')]$$

La utilidad óptima de un estado es la mejor acción disponible, promediada sobre los posibles estados siguientes ponderados por su probabilidad, sumando la recompensa inmediata más el valor descontado de continuar óptimamente.

2.3. Algoritmos de solución

Algoritmo	Idea
Value Iteration	Aplica la ecuación de Bellman como regla de actualización repetidamente sobre $U(s)$ para todo s , hasta que los valores cambien menos que un umbral ϵ . Al converger, $\pi^*(s) = \arg \max_a \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma U(s')]$.
Policy Iteration	Alterna entre (1) evaluación de política : resolver el sistema lineal de $U^\pi(s)$ para la política fija π , y (2) mejora de política : actualizar $\pi(s)$ eligiendo la acción que maximiza la utilidad usando la U^π recién calculada. Converge cuando la política deja de cambiar.

Cuadro 2: Comparación de los dos algoritmos clásicos para resolver un MDP.

Nota 2.1. Ambos algoritmos convergen a la misma política óptima π^* . Value iteration hace actualizaciones más baratas pero puede necesitar muchas iteraciones; policy iteration hace iteraciones más caras (resolver un sistema lineal) pero típicamente converge en menos pasos.

2.4. Ejercicio: RobotExplorador

Un robot aspiradora se mueve en un pasillo de tres estados $s_1 \rightarrow s_2 \rightarrow s_3$, donde s_3 es la estación de carga (estado **terminal**, recompensa +10). Desde s_1 o s_2 el robot ejecuta la acción *avanzar*: con probabilidad 0,8 se mueve al siguiente estado del pasillo, y con probabilidad 0,2 una rueda patina y se queda en el mismo estado. Cada transición no terminal cuesta $R = -1$ (consumo de batería). Use $\gamma = 0,9$.

Después de una iteración de value iteration se tienen las estimaciones $U(s_1) = 0$, $U(s_2) = 5$, $U(s_3) = 10$ (fijo, por ser terminal).

1. Escriba la ecuación de Bellman para $U(s_2)$ considerando la acción *avanzar*.

2. Calcule el nuevo valor de $U(s_2)$ usando las estimaciones de la iteración anterior.
3. Calcule el nuevo valor de $U(s_1)$ de la misma forma.
4. Sin volver a iterar, ¿por qué $U(s_1)$ tarda más iteraciones en reflejar la recompensa de s_3 que $U(s_2)$?

3. CSP: Repaso rápido

Nota

El detalle completo de estas técnicas (con ejemplos y el ejercicio del mapa de Australia) está en las notas de la Semana 5. Esta es solo una tabla de repaso rápido para el examen.

Técnica	Idea clave
AC-3 (consistencia de arcos)	Elimina de cada dominio los valores sin soporte en el vecino; si el dominio de una variable cambia, se vuelven a revisar sus vecinos (propagación en cadena). Se corre dentro de backtracking, no lo reemplaza: no garantiza encontrar la solución por sí sola.
Forward Checking	Al asignar $X_i \leftarrow v$, poda solo los dominios de los vecinos directos de X_i , una única vez. Más barato que AC-3 pero detecta menos inconsistencias por adelantado.
MRV + heurística de grado	Orden de variables : elegir la de dominio más pequeño (Minimum Remaining Values); en empate, la que más restricciones tiene sobre variables aún sin asignar (grado).
LCV	Orden de valores : elegir el valor que menos elimine opciones de los vecinos sin asignar.
Estructura de árbol	Si el grafo de restricciones no tiene ciclos, se resuelve en $\mathcal{O}(nd^2)$ sin backtracking (arco-consistencia hacia la raíz, luego asignación hacia las hojas).
Cutset conditioning	Para grafos con ciclos: fijar un conjunto pequeño de variables (cutset) para que el resto quede en forma de árbol; costo $\mathcal{O}(d^c \cdot (n-c)d^2)$, exponencial en el tamaño c del cutset.
Min-conflicts	Búsqueda local: parte de una asignación completa (con conflictos) y reasigna variables en conflicto al valor que menos conflictos genere. Rápido en la práctica, pero incompleto (puede atascarse).

Cuadro 3: Repaso rápido de técnicas de propagación y búsqueda para CSP.

Ejercicio 3.1. Para cada escenario, indique qué técnica de la tabla anterior es más adecuada y por qué:

1. Un CSP grande donde ya se sabe que el grafo de restricciones no tiene ciclos.
2. Un CSP con miles de variables (como n-reinas a gran escala) donde no importa perder completitud a cambio de velocidad.

3. Un paso de backtracking donde se quiere podar dominios de forma barata sin propagar en cadena.